МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА
ФАКУЛЬТЕТ ЕЛЕКТРОНІКИ ТА КОМП'ЮТЕРНИХ ТЕХНОЛОГІЙ

ЗВІТ

З ЛАБОРАТОРНОЇ РОБОТИ №1

з дисципліни "Функціональне програмування"

на тему "ОСНОВИ ФУНКЦІОНАЛЬНОГО ПРОГРАМУВАННЯ У PYTHON"

Варіант 12. Складський облік

Виконав:
Студент групи ФЕП-23с
Малецький Маркіян Володимирович

Перевірив:
проф. Свелеба С. А.

Львів – 2026

Мета: зрозуміти, що таке чиста функція і референтна прозорість; навчитися виявляти та ізолювати побічні ефекти; переписати імперативний фрагмент у функціональному стилі; застосувати `typing.Callable` для параметризації обчислень.

Теоретичні відомості: Чиста функція є детермінованою (однаковий вхід надає тотожний вихід), не мутує вхідні дані та глобальний стан і не виконує операцій вводу/виводу усередині. Референтна прозорість гарантує, що вираз у програмі можна замінити його обчисленим значенням без зміни поведінки системи. Побічні ефекти (I/O, виклики `print`, читання файлів, залежність від часу чи системного стану) виносяться в зовнішню оболонку (`app.py`), залишаючи розрахункове ядро (`core.py`) повністю чистим.

Хід роботи:

1. Для Варіанта 12 (Складський облік) сформував чисте функціональне ядро у `core.py`. Визначив типи `StockItem`, `RestockOrder` та `RestockReport` на основі `TypedDict`, а також аліаси функціональних типів `typing.Callable`:

```
# core.py
from collections.abc import Callable, Iterable
from typing import TypedDict, TypeVar

A = TypeVar("A"); B = TypeVar("B"); C = TypeVar("C")

class StockItem(TypedDict):
    id: int
    name: str
    stock: int
    price: float
    min_stock: int
    category: str

class RestockOrder(TypedDict):
    id: int
    name: str
    category: str
    order_qty: int
    unit_price: float
    base_cost: float
    discounted_cost: float
    final_cost: float

class RestockReport(TypedDict):
    items_to_order_count: int
    total_units_to_order: int
    total_budget_needed: float
    orders: list[RestockOrder]

NeedRestockFn = Callable[[StockItem], bool]
RestockQtyFn = Callable[[StockItem], int]
DiscountFn = Callable[[float, int], float]
TaxFn = Callable[[float], float]
ProcessorFn = Callable[[Iterable[StockItem]], RestockReport]

def compose(f: Callable[[B], C], g: Callable[[A], B]) -> Callable[[A], C]:
    return lambda x: f(g(x))

def is_below_minimum(item: StockItem) -> bool:
    return item["stock"] < item["min_stock"]

def calc_restock_qty_to_min(item: StockItem) -> int:
    return max(0, item["min_stock"] - item["stock"])

def calc_bulk_discount(base_cost: float, qty: int) -> float:
    if qty >= 20: return base_cost * 0.90
    if qty >= 10: return base_cost * 0.95
    return base_cost

def apply_standard_vat(amount: float) -> float:
    return amount * 1.20
```

2. У `core.py` реалізував побудову нових об'єктів без мутації входів та фабрику вищого порядку `make_restock_processor` для параметризації обчислень зваженими політиками:

```
def build_restock_order(
    item: StockItem, qty: int, apply_discount: DiscountFn, apply_tax: TaxF
n
) -> RestockOrder:
    base = round(qty * item["price"], 2)
    discounted = round(apply_discount(base, qty), 2)
    final = round(apply_tax(discounted), 2)
    return {
        "id": item["id"], "name": item["name"], "category": item["categor
y"],
        "order_qty": qty, "unit_price": item["price"], "base_cost": base,
        "discounted_cost": discounted, "final_cost": final
    }

def make_restock_processor(
    *, should_restock: NeedRestockFn, calc_qty: RestockQtyFn,
    discount_policy: DiscountFn, tax_policy: TaxFn
) -> ProcessorFn:
    def process(items: Iterable[StockItem]) -> RestockReport:
        filtered = [item for item in items if should_restock(item)]
        orders = [
            build_restock_order(item, calc_qty(item), discount_policy, tax
_policy)
            for item in filtered
        ]
        return {
            "items_to_order_count": len(orders),
            "total_units_to_order": sum(o["order_qty"] for o in orders),
            "total_budget_needed": round(sum(o["final_cost"] for o in orde
rs), 2),
            "orders": orders,
        }
    return process
```

3. У файлі `app.py` ізолював побічні ефекти (I/O, форматування та генерацію часу через інжекцію залежностей):

```
# app.py
from collections.abc import Callable
import time
from core import StockItem, RestockReport, make_restock_processor

def get_current_timestamp(time_provider: Callable[[], float] = time.time)
-> str:
    return time.strftime("%Y-%m-%d %H:%M:%S", time.localtime(time_provider
()))

def format_report(report: RestockReport, timestamp: str) -> str:
    lines = ["=" * 60, f"ЗВІТ З ДОЗАМОВЛЕННЯ СКЛАДСЬКИХ ПОЗИЦІЙ ({timestam
p})", "=" * 60]
    for o in report["orders"]:
        lines.append(f"{o['id']:<4} | {o['name']:<18} | {o['order_qty']:<
6} | {o['final_cost']:<12.2f}")
    lines.append(f"Необхідний бюджет: {report['total_budget_needed']:.2f}
грн")
    return "\n".join(lines)

def run_pipeline() -> None:
    warehouse_stock: list[StockItem] = [...] # вхідні дані
    processor = make_restock_processor(...)
    report = processor(warehouse_stock)
    print(format_report(report, get_current_timestamp()))
```

4. Створив головну точку входу у файлі `main.py`:

```
# main.py
from app import run_pipeline

def main() -> None:
    run_pipeline()

if __name__ == "__main__":
    main()
```

5. Написав pytest-тести у `tests/test_core.py` для перевірки референтної прозорості, відсутності мутації вхідних даних, правильності застосування `Callable`-політик та композиції функцій:

```
# tests/test_core.py
from copy import deepcopy
import pytest
from core import process_warehouse_pure, make_restock_processor, compose,
StockItem

def test_referential_transparency(sample_stock: list[StockItem]) -> None:
    res1 = process_warehouse_pure(sample_stock)
    res2 = process_warehouse_pure(sample_stock)
    assert res1 == res2

def test_no_input_mutation(sample_stock: list[StockItem]) -> None:
    snapshot = deepcopy(sample_stock)
    _ = process_warehouse_pure(sample_stock)
    assert sample_stock == snapshot

def test_custom_callable_policies(sample_stock: list[StockItem]) -> None:
    processor = make_restock_processor(
        should_restock=lambda it: it["stock"] < 10,
        calc_qty=lambda _: 10,
        discount_policy=lambda b, _: b * 0.8,
        tax_policy=lambda a: a
    )
    res = processor(sample_stock)
    assert res["items_to_order_count"] == 2
    assert res["total_budget_needed"] == 2240.0
```

6. Виконав запуск розробки, перевірив роботу програми, проходження усіх pytest-тестів, а також перевірив відповідність коду стандартам за допомогою статичних аналізаторів mypy, ruff та black:

```
      PS C:\functional-programming\Lab_01> python main.py ==================
========================================= ЗВІТ З ДОЗАМОВЛЕННЯ СКЛАДСЬКИХ ПОЗИЦІ
Й (2026-11-06 21:22:00) ==========================================================
====== ID | Назва | К-сть | Ціна | Разом (з ПДВ) ----------------------------
------------------------------ 101 | ESP32 NodeMCU | 11 | 180.00 | 2257.20 10
2 | Сервопривід SG90 | 22 | 65.00 | 1544.40 104 | Патч-корд RJ45 2м | 25 | 50.
00 | 1350.00 ------------------------------------------------------------- Всьо
го позицій на дозамовлення: 3 Загальна кількість одиниць: 58 Необхідний бюдже
т: 5151.60 грн ============================================================= PS
C:\functional-programming\Lab_01> python -m pytest -q ..... [100%] 5 passed in
0.08s PS C:\functional-programming\Lab_01> python -m mypy . Success: no issues
found in 4 source files PS C:\functional-programming\Lab_01> python -m ruff ch
eck . All checks passed! PS C:\functional-programming\Lab_01> python -m black
--check . All done! 4 files would be left unchanged.
```

Висновок: Під час виконання лабораторної роботи №1 я вивчив основні концепції функціонального програмування у Python на прикладі системи складського обліку (варіант 12). Отримано навички побудови чистого ядра (`core.py`) без побічних ефектів, глобальних змінних та мутацій вхідних об'єктів. Завдяки `typing.Callable` досягнуто гнучкої параметризації правил дозамовлення та розрахунку знижок/ПДВ через параметризовані політики. Побічні ефекти (вивід у консоль та отримання часу) ізольовані у файлах `app.py` та `main.py`. Написані pytest-тести підтвердили референтну прозорість та відсутність побічних мутацій даних.